

UE Apprentissage SIA S9

Supervise Learning

Mathieu FAUVEL

CESBIO, Université de Toulouse, CNES/CNRS/INRAe/IRD/UPS, Toulouse, FRANCE

August 31, 2023

Outline

Introduction

Introductory Example

Multi layer Perceptron

Convolutional Neural Networks

Transformer

- Contact: mathieu.fauvel@inrae.fr
- CV:

2004-2007 Ph.D. degree in Signal and Image Processing from the INP, Grenoble & the University of Iceland

2007-2008 Assistant Professor Grenoble

2008-2010 Post-doc position at INRIA - MISTIS Team

2010-2011 Assistant Professor Toulouse

2011-2018 Associate Professor at DYNAFOR & INP, Toulouse

Since 2018 Research (CRCN) at CESBIO, INRAe

- Research interests are:

Machine learning for environmental/ecological monitoring

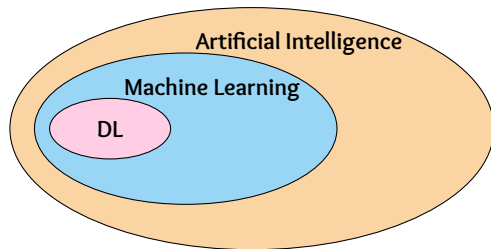
Course Objectives

TBC

What is Supervised (Machine) Learning ?

- **Artificial Intelligence**

Perform human tasks using computers and algorithms



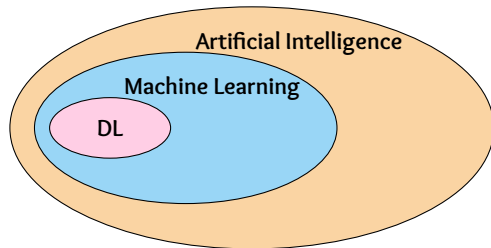
What is Supervised (Machine) Learning ?

- **Artificial Intelligence**

Perform human tasks using computers and algorithms

- Mitchell, *The discipline of machine learning*:

Machine Learning is defined as the capacity of a computer program to improve its performance measure with observations



What is Supervised (Machine) Learning ?

- **Artificial Intelligence**

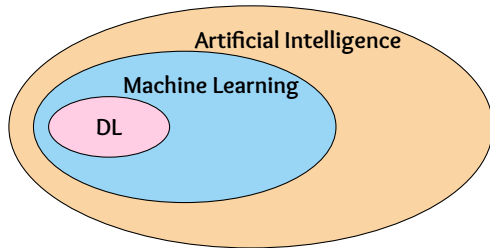
Perform human tasks using computers and algorithms

- Mitchell, *The discipline of machine learning:*

Machine Learning is defined as the capacity of a computer program to improve its performance measure with observations

- **Deep Learning**

- ★ Lot of data data
- ★ Complex model
- ★ High performance computing



Main Notations

Data

- Observed data $\mathbf{x} \in \mathbb{R}^d$, called *input variables* or *predictors*.
- Data to be predicted $y \in \mathbb{R}^p$, called *output variables* or *responses*.

Main Notations

Data

- Observed data $\mathbf{x} \in \mathbb{R}^d$, called *input variables* or *predictors*.
- Data to be predicted $y \in \mathbb{R}^p$, called *output variables* or *responses*.

Learning Problem

- If y are quantitative data: *regression* and $y \in \mathbb{R}^p$.
- If y are categorical data: *classification* and $y \in \{C_1, \dots, C_C\}$ with C_i refers to the i^{th} category.

Main Notations

Data

- Observed data $\mathbf{x} \in \mathbb{R}^d$, called *input variables* or *predictors*.
- Data to be predicted $y \in \mathbb{R}^p$, called *output variables* or *responses*.

Learning Problem

- If y are quantitative data: *regression* and $y \in \mathbb{R}^p$.
- If y are categorical data: *classification* and $y \in \{C_1, \dots, C_C\}$ with C_i refers to the i^{th} category.

Prediction function

$$\begin{aligned} f_{\theta} : \mathbb{R}^d &\rightarrow \mathbb{R}^p \\ \mathbf{x} &\mapsto y \end{aligned}$$

Online References

- Aston Zhang et al. “Dive into Deep Learning”. In: *arXiv preprint arXiv:2106.11342* (2021)
- Simon J.D. Prince. *Understanding Deep Learning*. MIT Press, 2023. URL: <https://udlbook.github.io/udlbook/>
- Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL: <http://probml.github.io/book1>
- Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL: <http://probml.github.io/book2>

Outline

Introduction

Introductory Example

- Linear model for regression

- Linear model for classification

Multi layer Perceptron

Convolutional Neural Networks

Transformer

Univariate linear model

- $f(x) = wx + b$ and $\theta = (w, b)$
- Loss function: $\ell(f(x_i), y_i) = (f(x_i) - y_i)^2$
- Objective function:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell_i$$

- Gradients update: $\theta^{t+1} = \theta^t - \eta \nabla_{\theta} \mathcal{L}$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{n} \sum_{i=1}^n x_i (wx_i + b - y_i)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n} \sum_{i=1}^n (wx_i + b - y_i)$$

Univariate linear model

- $f(x) = wx + b$ and $\theta = (w, b)$
- Loss function: $\ell(f(x_i), y_i) = (f(x_i) - y_i)^2$
- Objective function:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell_i$$

- Gradients update: $\theta^{t+1} = \theta^t - \eta \nabla_{\theta} \mathcal{L}$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{n} \sum_{i=1}^n x_i (wx_i + b - y_i)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n} \sum_{i=1}^n (wx_i + b - y_i)$$

Work

Implement the 1D regression in pytorch of the following function:

$$f(x) = 2x - 0$$

Do the notebook:

[regression_1d_toy.ipynb](#)

Multivariate linear model

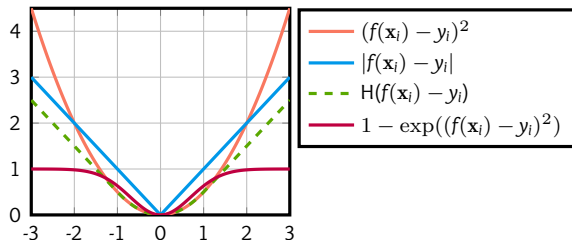
- $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ and $\boldsymbol{\theta} = (\mathbf{w}, b)$
- Same loss and objective function and so same gradient updates ...

with H stands for *Hubert loss*:

$$H(f(\mathbf{x}_i) - y_i) = \begin{cases} \frac{1}{2}(f(\mathbf{x}_i) - y_i)^2 & \text{for } |f(\mathbf{x}_i) - y_i| \leq \delta, \\ (|f(\mathbf{x}_i) - y_i| - 0.5), & \text{otherwise.} \end{cases}$$

Multivariate linear model

- $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ and $\boldsymbol{\theta} = (\mathbf{w}, b)$
- Same loss and objective function and so same gradient updates ...
- We can use other loss function

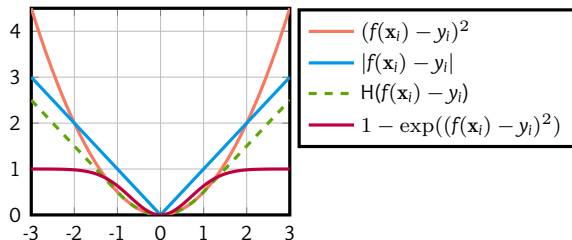


with H stands for *Hubert loss*:

$$H(f(\mathbf{x}_i) - y_i) = \begin{cases} \frac{1}{2}(f(\mathbf{x}_i) - y_i)^2 & \text{for } |f(\mathbf{x}_i) - y_i| \leq \delta, \\ (|f(\mathbf{x}_i) - y_i| - 0.5), & \text{otherwise.} \end{cases}$$

Multivariate linear model

- $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ and $\boldsymbol{\theta} = (\mathbf{w}, b)$
- Same loss and objective function and so same gradient updates ...
- We can use other loss function



with H stands for *Hubert loss*:

$$H(f(\mathbf{x}_i) - y_i) = \begin{cases} \frac{1}{2}(f(\mathbf{x}_i) - y_i)^2 & \text{for } |f(\mathbf{x}_i) - y_i| \leq \delta, \\ (|f(\mathbf{x}_i) - y_i| - 0.5), & \text{otherwise.} \end{cases}$$

Work

On the **California housing** data set

- Prepare (train/validation) data
- Define a multi. linear model
- Optimize your hyperparameters (batch size, learning rate)
- Switch L2 to other loss function

Scaling feature for multivariate linear model with L2 loss function

- Suppose we have two features of different scale (e.g. because of different unit)

$$\mathbf{x}_1 \sim \mathcal{N}(0, 1) \text{ and } \mathbf{x}_2 \sim \mathcal{N}(10, 10)$$

- What is the impact on the gradient update ?

Scaling feature for multivariate linear model with L2 loss function

- Suppose we have two features of different scale (e.g. because of different unit)

$$\mathbf{x}_1 \sim \mathcal{N}(0, 1) \text{ and } \mathbf{x}_2 \sim \mathcal{N}(10, 10)$$

- What is the impact on the gradient update ?
- Noting $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top$ and $\mathbf{y} = [y_1, \dots, y_n]^\top$

$$\nabla_{\mathbf{w}} = -\mathbf{X}^\top \underbrace{(\mathbf{y} - \mathbf{X}\mathbf{w})}_{\mathbf{e}_i} \Rightarrow \nabla_{\mathbf{w}_p} = -\sum_{i=1}^n \mathbf{e}_i \mathbf{x}_{ip}$$

- Parameter update

$$\mathbf{w}_p^{(t+1)} = \mathbf{w}_p^{(t)} + \frac{\eta}{n} \sum_{i=1}^n \mathbf{e}_i^{(t)} \mathbf{x}_{ip}$$

Scaling feature for multivariate linear model with L2 loss function

- Suppose we have two features of different scale (e.g. because of different unit)

$$\mathbf{x}_1 \sim \mathcal{N}(0, 1) \text{ and } \mathbf{x}_2 \sim \mathcal{N}(10, 10)$$

- What is the impact on the gradient update ?
- Noting $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top$ and $\mathbf{y} = [y_1, \dots, y_n]^\top$

$$\nabla_{\mathbf{w}} = -\mathbf{X}^\top \underbrace{(\mathbf{y} - \mathbf{X}\mathbf{w})}_{\mathbf{e}_i} \Rightarrow \nabla_{\mathbf{w}_p} = -\sum_{i=1}^n \mathbf{e}_i \mathbf{x}_{ip}$$

- Parameter update

$$\mathbf{w}_p^{(t+1)} = \mathbf{w}_p^{(t)} + \frac{\eta}{n} \sum_{i=1}^n \mathbf{e}_i^{(t)} \mathbf{x}_{ip}$$

- Standardization of the input feature (and for regression the output too): [scaling_feature.ipynb](#)

$$\tilde{\mathbf{x}} = (\mathbf{x} - \boldsymbol{\mu}) \oslash \boldsymbol{\sigma}$$

Outline

Introduction

Introductory Example

Linear model for regression

Linear model for classification

Multi layer Perceptron

Convolutional Neural Networks

Transformer

Classification as a regression problem

- Predict categorical variables ("Dogs", "Cats", "Monkey" ...)
- Estimation of the posterior probability

$$p(y_i = c | \mathbf{x}_i) \forall c \in \{1, \dots, C\} \text{ with } p(y_i = c | \mathbf{x}_i) \geq 0 \text{ and } \sum_{c=1}^C p(y_i = c | \mathbf{x}_i) = 1$$

- MAP rule: $\hat{c}_i = \arg \max_c p(y_i = c | \mathbf{x}_i)$

Classification as a regression problem

- Predict categorical variables ("Dogs", "Cats", "Monkey" ...)
- Estimation of the posterior probability

$$p(y_i = c | \mathbf{x}_i) \forall c \in \{1, \dots, C\} \text{ with } p(y_i = c | \mathbf{x}_i) \geq 0 \text{ and } \sum_{c=1}^C p(y_i = c | \mathbf{x}_i) = 1$$

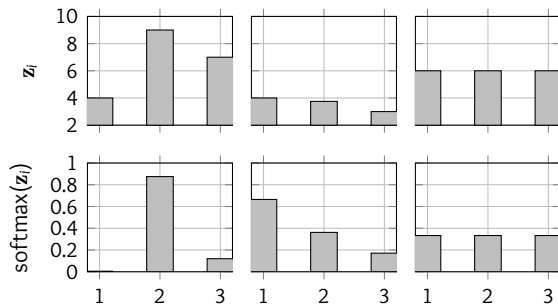
- MAP rule: $\hat{c}_i = \arg \max_c p(y_i = c | \mathbf{x}_i)$
- Softmax linear regression

$$1 = \sum_{c=1}^C p(y_i = c | \mathbf{x}_i) = \sum_{c=1}^C \exp \left\{ \mathbf{w}_c^\top \mathbf{x}_i + b_c + K_i \right\} = \exp \{K_i\} \sum_{c=1}^C \exp \left\{ \mathbf{w}_c^\top \mathbf{x}_i + b_c \right\}$$

thus

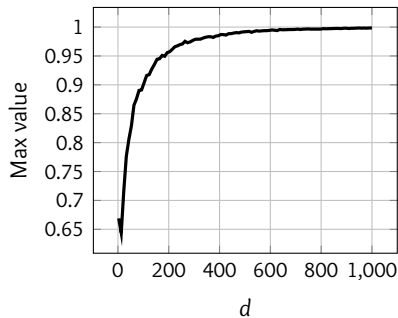
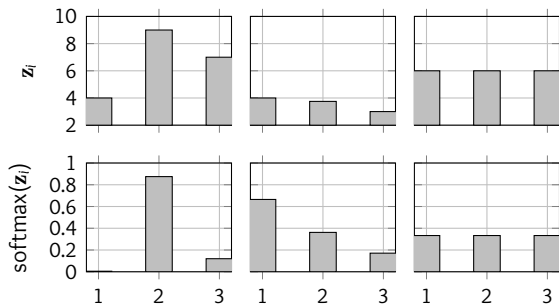
$$p(y_i = c | \mathbf{x}_i) = \frac{\exp \left\{ \mathbf{w}_c^\top \mathbf{x}_i + b_c \right\}}{\sum_{c'=1}^C \exp \left\{ \mathbf{w}_{c'}^\top \mathbf{x}_i + b_{c'} \right\}} = \frac{\exp(\mathbf{z}_{ic})}{\sum_{c'=1}^C \exp(\mathbf{z}_{ic'})} = \text{softmax}(\mathbf{z}_i)_c$$

Softmax



- Translation invariant: $\text{softmax}(\mathbf{z} + \mathbf{K}) = \text{softmax}(\mathbf{z})$
- Preserve ordering relation: $\arg \max_c(\mathbf{p}_i) = \arg \max_c(\mathbf{z}_i)$
- If $\mathbf{z} \in \mathbb{R}^d$ with $d \rightarrow \infty$ then $\text{softmax}(\mathbf{z}) \rightarrow \text{"One-hot-vector"}$

Softmax



- Translation invariant: $\text{softmax}(\mathbf{z} + K) = \text{softmax}(\mathbf{z})$
- Preserve ordering relation: $\arg \max_c(\mathbf{p}_i) = \arg \max_c(\mathbf{z}_i)$
- If $\mathbf{z} \in \mathbb{R}^d$ with $d \rightarrow \infty$ then $\text{softmax}(\mathbf{z}) \rightarrow$ “One-hot-vector”

Cross entropy loss function

- One hot encoding: $y_i = c \rightarrow \mathbf{y}_i \in \mathbb{R}^C$, $\mathbf{y}_{ic} = 1$ if $i = c$ else 0
- Likelihood:

$$l(\mathbf{y}_i, \hat{\mathbf{y}}_i) = \prod_{c'=1}^C p(y_i = c' | \mathbf{x}_i)^{\mathbf{y}_{ic'}} = \frac{\exp(\mathbf{z}_{ic})}{\sum_{c'=1}^C \exp(\mathbf{z}_{ic'})}$$

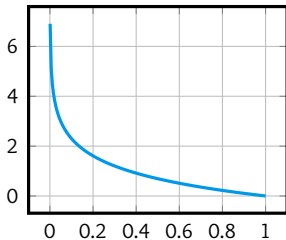
Cross entropy loss function

- One hot encoding: $y_i = c \rightarrow \mathbf{y}_i \in \mathbb{R}^C$, $y_{ic} = 1$ if $i = c$ else 0
- Likelihood:

$$l(\mathbf{y}_i, \hat{\mathbf{y}}_i) = \prod_{c'=1}^C p(y_i = c' | \mathbf{x}_i)^{y_{ic'}} = \frac{\exp(\mathbf{z}_{ic})}{\sum_{c'=1}^C \exp(\mathbf{z}_{ic'})}$$

- Negative log likelihood:

$$\begin{aligned}\ell(\mathbf{y}_i, \hat{\mathbf{y}}_i) &= - \sum_{c'=1}^C y_{ic'} \log \{p(y_i = c' | \mathbf{x}_i)\} \\ &= -\log \{p(y_i = c | \mathbf{x}_i)\} \\ &= -(\mathbf{w}_c^\top \mathbf{x}_i + b_c) + \underbrace{\log \left\{ \sum_{c'=1}^C \exp(\mathbf{w}_{c'}^\top \mathbf{x}_i + b_{c'}) \right\}}_{\text{logsumexp}(\mathbf{z}_i)}\end{aligned}$$



Work

What happens if:

- All $\mathbf{z}_{ic}$ are small ?
- One $\mathbf{z}_{ic}$ is very large ?
- All $\mathbf{z}_{ic}$ are very large ?

Multivariate linear classifier

- The model

$$\mathbf{p}_i = \text{softmax}(\mathbf{W}\mathbf{x}_i + \mathbf{b})$$

- Cross entropy loss function
- Same comments than for regression
 - ★ Split train, val and test
 - ★ Standardization
 - ★ Batch training
 - ★ Tune learning rate

Work

Implement a linear classifier in pytorch for the classification of **Fashion MNIST** data set.

Bibliography I

- [1] T. Mitchell. *The discipline of machine learning*. Tech. rep. CMU ML-06 108. 2006. URL: <http://ra.adm.cs.cmu.edu/anon/usr0/ftp/anon/ml/CMU-ML-06-108.pdf>.
- [2] Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL: <http://probml.github.io/book2>.
- [3] Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL: <http://probml.github.io/book1>.
- [4] Simon J.D. Prince. *Understanding Deep Learning*. MIT Press, 2023. URL: <https://udlbook.github.io/udlbook/>.
- [5] Aston Zhang et al. “Dive into Deep Learning”. In: *arXiv preprint arXiv:2106.11342* (2021).

This work is licensed under a [Creative Commons “Attribution-ShareAlike 4.0 International”](https://creativecommons.org/licenses/by-sa/4.0/) license.

